

Canonical Case File - structured-product suitability

Assessment 71386234-daae-4896-91fe-4c469cf59af2 - DIFC Wealth Partners
(DEMONSTRATION FIRM) - instrument XS2500000018 - verdict NOT SUITABLE

This is the complete record of one structured-product suitability assessment: who was assessed, against whose terms, what the zero-knowledge circuit answered, where that answer is recorded in two independent audit chains, and which regulatory obligations it bears on. It is assembled from a single call to the issuing server's evidence endpoint and, where supplied, the relying party's own audit log.

Assessment	71386234-daae-4896-91fe-4c469cf59af2
Firm	DIFC Wealth Partners (DEMONSTRATION FIRM)
Instrument	XS2500000018 5-year USD capital-protected note (DIFC-distributed)
VERDICT	NOT SUITABLE

Read section 2 before acting on the verdict. The single most consequential misreading available to a relying party is described there, and it is a misreading that leaves the reader holding cryptographic evidence which appears to support the wrong conclusion.

- Section 1, client and product metadata - including a plain statement of what the issuer does not hold.
 - Section 2, zero-knowledge proof verification receipt - the key, the inputs, the verdict.
 - Section 3, audit chains - Memtara's own log and the relying party's AIH00TS chain.
 - Section 4, regulatory mapping - DFSA Conduct of Business, and CBUAE.
 - Section 5, cryptographic seal - what it is, and what it deliberately is not.
-

1. Client and product metadata

1a. The subject

Client reference	97042d32-ab91-461b-9bae-9e0c62c99448
Assessment	71386234-daae-4896-91fe-4c469cf59af2
Circuit	wealth_suitability
Request status	verified
Opened at	2026-09-16T08:55:00Z
Firm	DIFC Wealth Partners (DEMONSTRATION FIRM) (bank)
Firm id	5c1d8e2f-7a90-4b1c-8d3e-2f4a6b8c0d1e

The subject is identified by the UUID above and by nothing else. Memtara holds no name for this person, so none appears here; the firm resolves the UUID against its own client records.

1b. What the issuer does not hold

Stated explicitly, because a reader of an evidence pack will look for the client's finances and should know why they will not find them rather than be left to wonder whether their absence is an oversight.

Memtara holds no annual income figure, no liquid-assets figure, no risk-tolerance score and no existing-holdings value for this subject. It never did. The four limbs of the assessment were evaluated inside a zero-knowledge

circuit on the holder's own device; the figures were private witnesses to that computation and were not transmitted. What crossed the wire was a proof and twelve public inputs, of which the last is the one-bit verdict.

This is the product, not a limitation of it. A firm that cannot produce the figures cannot leak, mis-share or be compelled to hand over figures it does not have, and the obligations that attach to holding them do not attach here. The consequence a reader should also register: nobody - not the firm, not Memtara, not this document - can reconstruct the client's finances from this pack, so a challenge to the verdict must be resolved by re-running the assessment, not by re-reading the file.

Attributes disclosed none (the verdict is not an attribute)
Issuer's own note Memtara holds no income, liquidity, risk-tolerance or holdings figure for this subject.

1c. The instrument

ISIN XS2500000018
Name 5-year USD capital-protected note (DIFC-distributed)
Registry id 3fa85f64-5717-4562-b3fc-2c963f66afa6

product_ref is the digest of the ISIN that the circuit actually consumed as public input 3. It is what binds this proof to this instrument; an ISIN printed on a report binds nothing.

product_ref 0x2b7e151628aed2a6abf7158809cf4f3c762e7160f38b4da56a784d9045190cfe

1d. The terms assessed against

Term	Value	Public input
Minimum annual income	500000	4 min_income
Minimum liquid assets	1000000	5 min_liquidity
Maximum post-trade concentration	30%	6 max_concentration_percent
Product risk level	3	7 product_risk_level

Provenance product registry, snapshotted when the assessment was opened

The provenance line is the load-bearing part of this subsection. These four numbers are the firm's, they come from the product registry, and they were snapshotted into the assessment when it was opened - before the client was asked anything. The snapshot matters: amending the product tomorrow does not change what this proof was measured against.

Every value of these thresholds yields a valid proof. A client who could choose its own minimum income could prove itself suitable for anything, with cryptography that checks out perfectly. What makes the verdict mean what the firm thinks it means is the server's comparison of each submitted public input against the terms above, not the proof.

1e. The assessment window

Window opens 2026-09-16T08:55:00Z
Window closes 2026-09-16T09:10:00Z
Grant expires 2026-09-16T09:10:00Z

The window is enforced inside the circuit, not only by the server: public input 0 must fall within it. The nonce below is single-use and was issued when the assessment was opened, which is what prevents a verdict being back-dated onto a recommendation that had already been made.

nonce 0x0f86d081884c7d659a2feaa0c55ad015a3bf4f1b2b0b822cd15d6c15b0f00a08

party that reads the exit code and stops there approves everyone who was assessed, including everyone who failed, while holding cryptographic evidence that appears to support it. This is demonstrated with a real Barretenberg proof in tests/test_wealth_suitability_e2e.py, not merely asserted.

The verdict is not the exit code. It is a public output of the circuit, read explicitly from its position in the public input vector:

```
VERDICT      NOT SUITABLE
read from    public input 11 of 12
assessed at  2026-09-16T09:00:41Z
assessed     true
```

Issuer's own note ``bb verify`` succeeding means the proof is well formed, not that the client passed. The verdict is the circuit's public output and is read explicitly.

2e. The attestation token

Supplied by the caller. Anyone can re-verify this token against the issuer's published JWKS - GET <issuer>/.well-known/jwks.json, then one EdDSA check in any JOSE library - with no call to Memtara, no cooperation from the firm, and no reliance on this document. That makes it the strongest authenticity claim in the pack, stronger than the seal in section 5.

This exporter decoded the token but did NOT verify its signature. Verification needs the issuer's key set, and a pack whose 'verified' stamp depended on what DNS answered at export time would be asserting something it cannot evidence. The check above is the reader's to run.

Algorithm	EdDSA
Key id (kid)	1IG2tMH7J2wbJZnOf8LJzQitKf7LMvoAElsuDMVM54Y
Signature	64 bytes

Decoded claims:

```
{
  "cbluae_clauses": [
    "5(c)",
    "5(d)",
    "4(a)"
  ],
  "circuit": "wealth_suitability",
  "dfsa_rules": [
    "COB 3.1"
  ],
  "exp": 1789456200,
  "iat": 1789455900,
  "iss": "https://issuer.memtara.example",
  "predicate": "structured_product_suitable",
  "proof_hash": "5a918a2496fc74d3089e97dc8d49e27e5454cd5e92cbfbbf78cc6388e172ab81",
  "sub": "97042d32-ab91-461b-9bae-9e0c62c99448",
  "suitable": false,
  "verified": true
}
```

Two claims are easy to conflate and must not be. 'verified' is about the proof - it is always true on an issued token, because a failed verification produces an error rather than a token. 'suitable' is the answer that proof carried, and it is legitimately false sometimes. A relying party that gates on 'verified' has gated on nothing.

The token as issued:

[illegible]

SkiXX0.Vaen5e2yR-BReTKwoQAXo0sXAxgp9w0SSPaImhbUVXXyhHWNpAkiAVut_-Rc010anUTTcAm-nMxXke0-MBP
wDw

The token had a lifetime of a few minutes and expired long before this pack was printed, so reproducing it here discloses no usable credential. It is reproduced because it is the artefact an examiner re-checks.

3. Audit chains

Two independent, append-only hash chains cover this assessment: Memtara's own `audit_log`, and the relying party's AIH00TS `audit.jsonl`. The systems never call each other. They are correlated by the proof digest and the `regulatory_audit_id` carried in the token, which is what lets an auditor holding both logs show they describe the same event - and what stops either operator fabricating a match without breaking a chain.

3a. Memtara `audit_log` excerpt

seq	Event	Created at	prev_hash	event_hash
4118	wealth_suitability_reques ted	2026-09-16T08:55:00Z	aa11bb22cc33dd44	c2b7d1a0f3e49568
4131	wealth_suitability_assess ed	2026-09-16T09:00:41Z	c2b7d1a0f3e49568	d3c8e2b1a4f50679

Hashes are shown truncated to 16 characters for the table; the full values follow. `seq` is the position in the firm's global chain, so consecutive entries for this assessment are not expected to be adjacent - other assessments were interleaved.

```
seq 4118 wealth_suitability_requested
event_hash c2b7d1a0f3e4956871209a3b4c5d6e7f8091a2b3c4d5e6f70819a2b3c4d5e6f7
prev_hash aa11bb22cc33dd44ee55ff6677889900112233445566778899aabbccddeeff00
seq 4131 wealth_suitability_assessed
event_hash d3c8e2b1a4f5067982310b4c5d6e7f8091a2b3c4d5e6f708192a3b4c5d6e7f80
prev_hash c2b7d1a0f3e4956871209a3b4c5d6e7f8091a2b3c4d5e6f70819a2b3c4d5e6f7
```

3b. AIH00TS SHA-256 audit chain

```
Log cro_demo/aihoots_audit.jsonl
Records read 1
Entries matching this assessment 0
```

No entry in the supplied log carries a proof digest from this assessment. Either this is the wrong log, or the relying party never processed this assessment through its gateway. The linkage is by `proof_hash`: the digests in section 2b are what an entry must carry to appear here.

Proof digests from section 2b with no matching entry in this log (expected for rejected attempts, which never reach a relying party):

```
5a918a2496fc74d3089e97dc8d49e27e5454cd5e92cbfbbf78cc6388e172ab81
```

3c. audit-verify verdict

Run as a subprocess against AIH00TS's own pinned code, reproduced verbatim - exit code and both streams. Nothing here is this tool's paraphrase. A line wider than the page is clipped rather than re-flowed, with a `>` marking the cut; the untruncated text is part of the canonical evidence hashed in section 5.

```
$ python -m src.verifier.cli cro_demo/aihoots_audit.jsonl
exit code: 0
--- stdout ---
OK: audit chain intact (/Users/prashantsingh/Desktop/memtara-zkp/cro_demo/aihoots_audit.jsonl)
```

Exit code 0 means every record's stored hash matches its recomputed contents and every `prev_hash` matches its predecessor. It means the log is internally consistent; it does not mean the log is complete. A hash chain proves nothing was altered, not that nothing was deleted from the end or never written - retention and off-box replication remain the firm's obligation.

4. Regulatory mapping

Neither Memtara nor the gateway is a Licensed Financial Institution or an Authorised Firm. Every obligation below binds the firm. What these systems supply is evidence - the artefact the firm hands its Audit and Risk Committee, or a regulator, to show a control was operating rather than asserted. This section maps what this pack evidences and is explicit about what it does not.

4a. DFSA Conduct of Business - the citation, both halves

Claim value emitted in Memtara tokens	COB 3.1
Correct citation for a DFSA filing	COB 3.4 (Suitability; the assessment at COB 3.4.2)
What COB 3.1 actually is	Application

The discrepancy, in one sentence: Memtara's tokens and audit records emit the string COB 3.1 because that is the identifier the relying parties integrating against this system were told to match, while the rule this assessment actually discharges is COB 3.4 (Suitability) - COB 3.1 is titled Application - so a deployment filing evidence with the DFSA should cite COB 3.4 and treat the claim value as an integration contract rather than as a citation.

Changing the emitted string unilaterally would break every relying party matching on it, which is why it has not been changed silently. It is a free-text list and one line changes it; the decision belongs to the firm, not to the vendor. This is recorded the same way in docs/REGULATORY_MATRIX.md.

A caveat that applies to this section and not to the CBUAE one below: the DFSA Rulebook is NOT vendored in this repository, and its current rule text was not retrievable in a form that could be quoted. The section titles are confirmed; the substance is taken from an archived version (COB/VER7/08-06, S.6.2.1) whose wording may since have changed. Read the mapping as being against the substance of a suitability obligation and verify the current text before relying on it.

Obligation	What this pack shows	Remains the firm's
A recommendation must be suitable, having regard to objectives and risk tolerance.	risk_tolerance >= product_risk_level is one of the four limbs, evaluated in-circuit against the registered risk level.	Investment objectives are not modelled by the circuit, and any relevant fact outside the four limbs still has to be acted on.
The assessment must precede the recommendation.	The terms were registered and the nonce issued before the client was asked; the token is minted only after a proof against those terms verifies.	Not making recommendations outside the gated channel.
The firm must be able to demonstrate it assessed.	This document, re-verifiable years later against the published verification key and JWKS without Memtara or the firm.	Retaining both logs for the applicable period.
A decline must be evidenced as much as an approval.	suitable is a public output, not an assertion, so a proof of NOT SUITABLE exists and is attested identically.	Acting on the decline.
The criteria must be the firm's, not the client's.	Section 1d: terms from the product registry, snapshotted at open, compared against every submitted public input.	Maintaining the product master.

Obligation	What this pack shows	Remains the firm's
Records must be retained (six years in the archived version).	Not covered. A hash chain proves internal consistency, not availability - an operator can still delete a whole log.	Retention, off-box replication, append-only storage.

One limit worth naming while a reader is looking at the four limbs: the rule enumerates objectives and risk tolerance. It does not mandate an income floor, a liquidity floor or a concentration cap - those are the firm's own suitability policy, which is the correct shape for a rule that delegates the criteria to the firm. The circuit implements a policy, and this pack evidences that the policy was applied before the recommendation and can be shown.

4b. CBUAE Guidance Note

Guidance Note on Consumer Protection and the Responsible Adoption and Use of Artificial Intelligence and Machine Learning by Licensed Financial Institutions in the U.A.E. (CBUAE_EN_6958_VER1). Unlike the DFSA rulebook, this document is vendored in the repository and every clause below was read from it. Clause identifiers follow the document's own scheme - numbered sections, lettered sub-clauses; there is no section 4.3 or 5.2 despite what some briefs assume.

Clause	What it requires	What this pack shows
S.5(c)	Personal data used only for legitimate and proportionate purposes; in-country retention.	Proportionality is enforced by the circuit rather than by policy. The firm received one boolean; storage and residency duties do not attach to figures it never held.
S.5(d)	Privacy-by-design and security-by-design built into AI systems.	The plaintext is not in the system to protect. Single-use nonce, in-circuit time bound, and a vault root the server pins rather than accepts.
S.4(a)	Be transparent about AI use and high-impact decisions, and be able to disclose how they were made.	This pack is that disclosure: the terms, the inputs, the verdict and the position it was read from.

The gaps a reader should carry away, unchanged from the matrix: bias measurement (S.3(a)) and training-data representativeness (S.3(b)) are not covered by either system; bilingual disclosure (S.4(b)) is specified but not built; the complaints and redress half of S.7(c) does not exist in either system; and the proof token is not revocable inside its short lifetime, which is a deliberate trade for offline validation rather than an oversight. Nothing in this section is a legal opinion or a substitute for the firm's own compliance assessment.

5. Cryptographic seal

5a. Digest of the evidence

Everything this pack asserts is also serialised as canonical JSON - sorted keys, no insignificant whitespace, the same convention the AIH00TS chain uses for its record hashes - and hashed. Recomputing this digest from a fresh export of the same assessment reproduces it exactly; a different value means the underlying record moved.

```
canonical evidence SHA-256 32740f0be3df9377263d3300fe7419d1f187164e0d3263b9e3a92e1dea7ad261
```

The digest deliberately excludes the export timestamp and the tool version, so it identifies the evidence rather than the run that printed it. Those two live in the sidecar.

5b. Digest of this document

The SHA-256 of the finished PDF bytes cannot appear inside those same bytes - a document cannot contain its own digest. It is written to the sidecar file alongside this one:

```
case_file.pdf.seal.json
```

To check this copy is the one that was sealed: `shasum -a 256 <this file>`, and compare with `pdf.sha256` in the sidecar. The exporter's own `verify` subcommand does the same thing and also re-checks the signature.

5c. What the seal is not

This is NOT a PAdES or AdES signature. It is not embedded in the PDF, it is not a qualified or advanced electronic signature under any framework, and no PDF viewer will show a green tick or a signature panel for it. Any reader who opens this file expecting the viewer to vouch for it will be told nothing, and that is the expected behaviour rather than a fault in the file.

Nor is the strongest claim in this pack the exporter's own. That is the Memtara-issued JWT of section 2e, verifiable against the issuer's published JWKS by anyone, with no call to Memtara and no cooperation from the firm - and behind it the proof itself, re-verifiable against the published verification key. The seal below covers the paper. Those cover the facts.

5d. Authentication status of this pack

This pack is UNSIGNED. No signing key was supplied to the exporter, so the pack is tamper-evident - the digests above will not match an altered copy - but unauthenticated: nothing here establishes who produced it.

The exporter deliberately did not generate a key of its own to sign with. An ephemeral signature verifies against a key nobody holds and means nothing, while looking exactly like a signature that means something. Re-run with `--signing-key` to sign with a key the firm actually controls.

5e. How to re-verify all of this without the firm

- Step 1. Check the token in section 2e against the issuer's published JWKS with any JOSE library. Prefer the pinned snapshot in an offline verification bundle (`jwks_snapshot.json`, see docs/VERIFY.md step 8) over a live fetch: a key set retrieved today evidences today's DNS, not this issuance, and a live fetch stops working the day the issuer or that key goes away.
- Step 2. Take the proof bytes and the public inputs from the firm's records and

run bb verify against the verification key whose digest is in section 2a, target noir-recursive.

- Step 3. Read public input 11. It is the verdict in section 2d. Do not read the exit code as the verdict.
- Step 4. Re-run AIH00TS's audit-verify over the relying party's log to confirm nothing moved, and match the entries by the proof digest in section 3b.
- Step 5. Compare the SHA-256 of this PDF with the sidecar, and the signature if there is one.

Step 2 is what distinguishes this from a signed PDF of a spreadsheet. The firm cannot produce a proof that verifies against that key unless the assessment really was carried out over a vault committed to the root recorded at the time - and neither can Memtara.